

# Classification of Protocols

**Ankit Srivastava**  
**EE2028**

Unless stated otherwise, for the second part of the module,

‘STM32 Chip’ = STM32L4S5VIT6

‘Board’ = B-L4S5I-IOT01A

Acknowledgement: Slides adapted from Dr Rajesh Panicker



# OVERVIEW

- What is a Protocol?
- Classification of Protocols

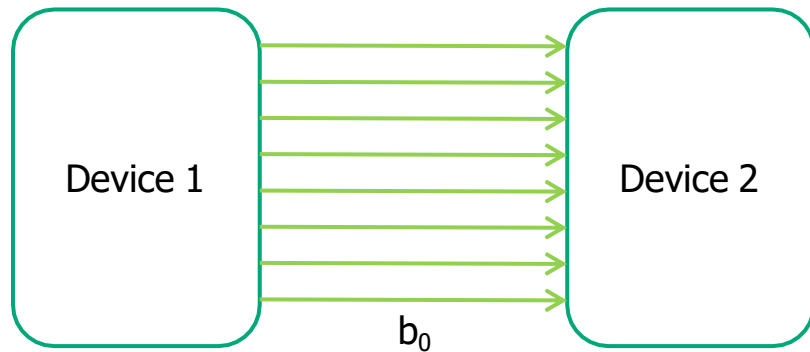
# WHAT IS A PROTOCOL?

- Voltage to be used
- Transmission data rate
- Transmit data with or without clock
- Transmission through single or multi-wire
- Selection of device

# CLASSIFICATION OF PROTOCOLS

- Parallel vs Serial
- Synchronous vs Asynchronous
- Bus-based vs Point-to-point
- Master-Slave vs Peer-to-peer
- Simplex vs Half-duplex vs Full-duplex
- In-band addressing vs Out-of-band addressing

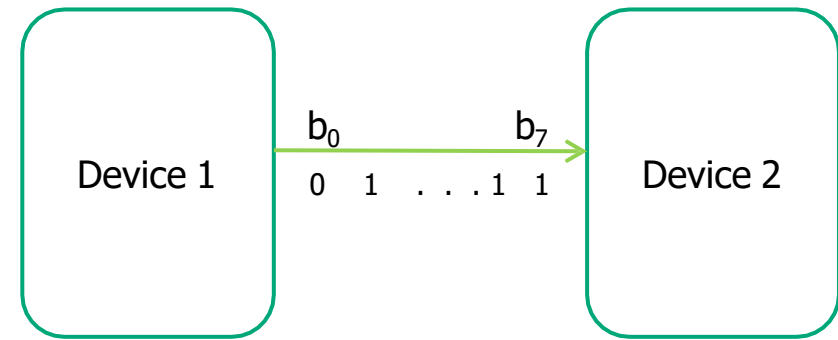
# PARALLEL VS SERIAL



Need not always be 8-bit

**Parallel:** use several parallel wires to transmit bits in parallel

- Processor-to-memory buses, old (pre-2005) protocols such as PCI, IDE, printer port, AXI, AXI-Stream



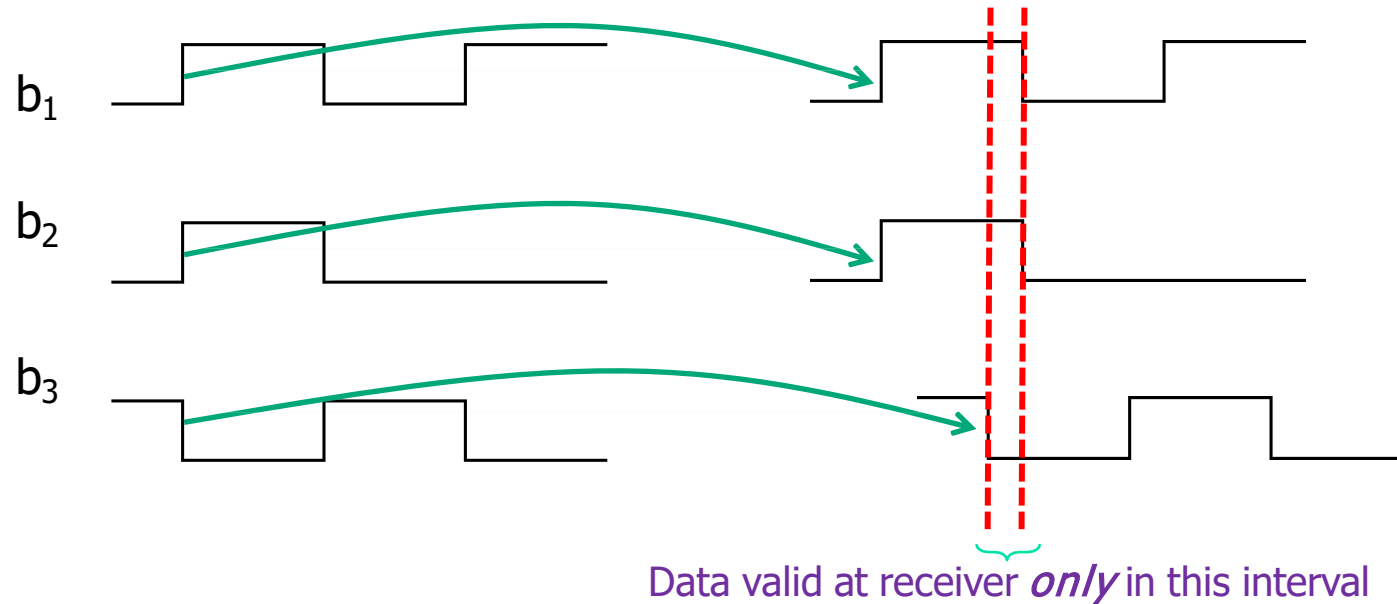
MSB First (Need not always be the case)

**Serial:** data is sent through a single wire, one bit at a time

- Serial ATA (SATA), PCIe, I2C, SPI, UART

Note: Data *processing* and *storage* (e.g. ALU, Registers, Memories) deal with data in a parallel manner even when data *transfer* is serial. This means serial protocols require a parallel-serial converter at the transmitter, serial-parallel converter at the receiver

# PARALLEL PROTOCOL ISSUES



## Parallel Protocols:

- suffers from bus-skew: different signals travel at different speeds
- suffers from cross-talk: signals from different wires interfere with each other

# SUMMARY: PARALLEL VS SERIAL

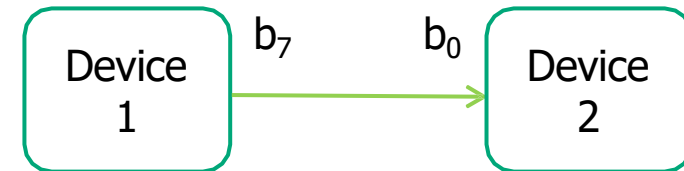
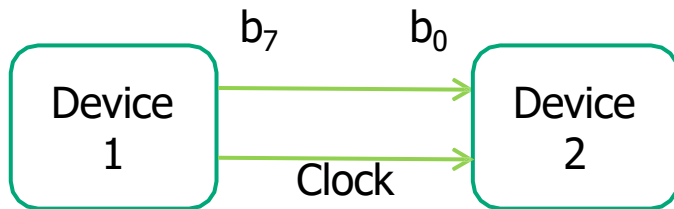
## Parallel Protocols:

- Used for shorter distance (preferred for few cms or below)
- Error-prone over longer distances due to skew & cross-talk
- Requires more hardware as multiple wires carry the information
- In-general, higher cost as more hardware is required

## Serial Protocols:

- Preferred for longer distances but can be used for smaller distances
- Less prone to skew & cross-talk
- Requires less hardware as single wire carries the information
- In-general, lower cost as less hardware is required

# SYNCHRONOUS VS ASYNCHRONOUS



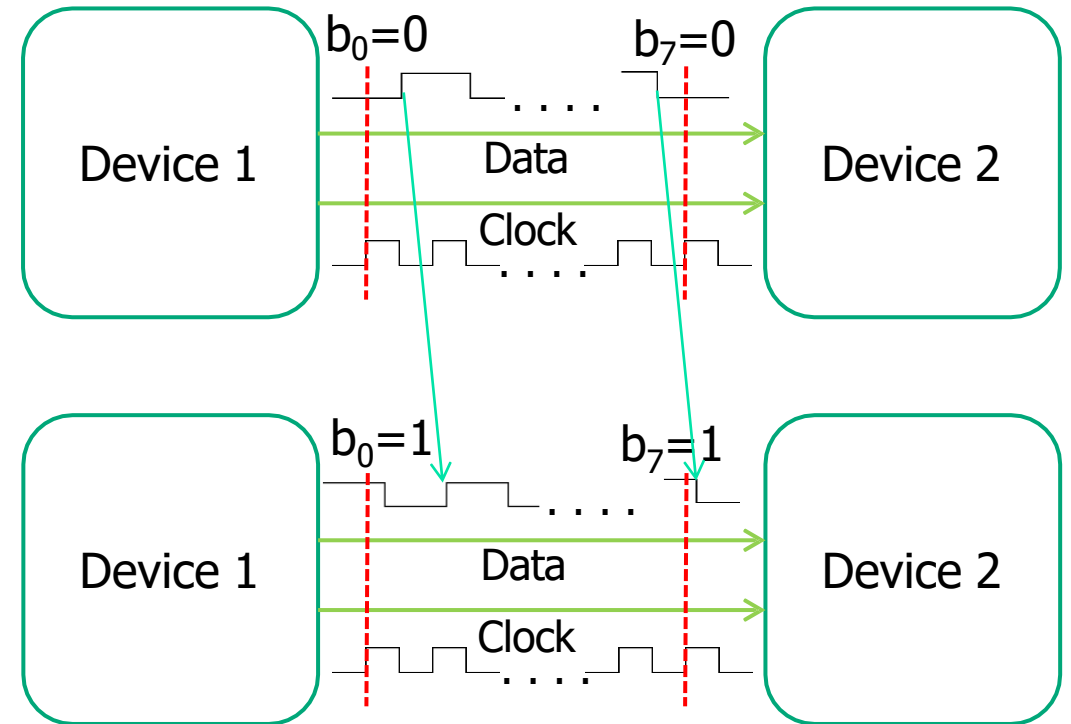
- **Synchronous:** one of the wires carry clock – allows transmitter and receiver to have a common time reference
- Synchronous protocols are generally faster
- I2C, SPI, AXI, AXI-Stream

- **Asynchronous:** only data is transmitted, not clock
- Asynchronous protocols are generally slower
- UART, USB



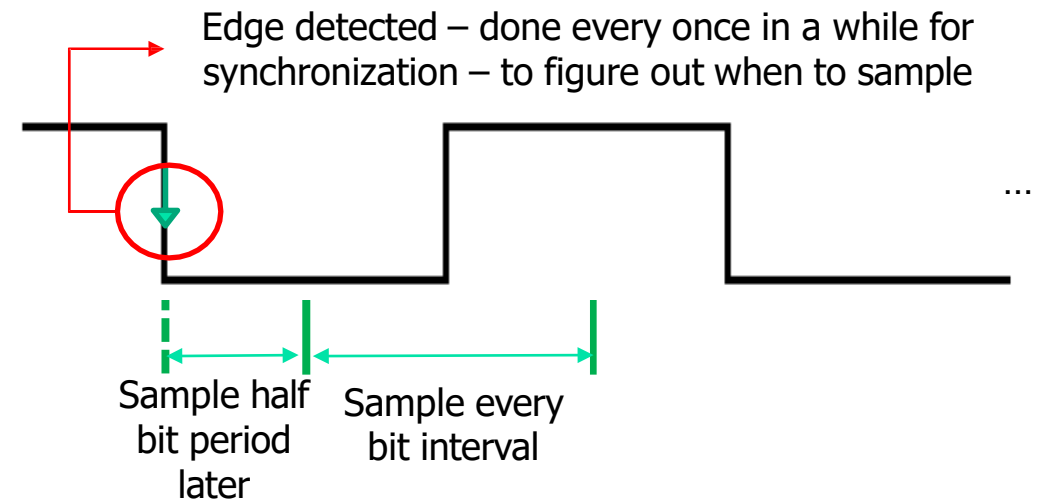
# LIMITATIONS OF SYNCHRONOUS PROTOCOLS

Synchronous protocols suffer from the same issue as the parallel protocols i.e. bus-skew and cross-talk



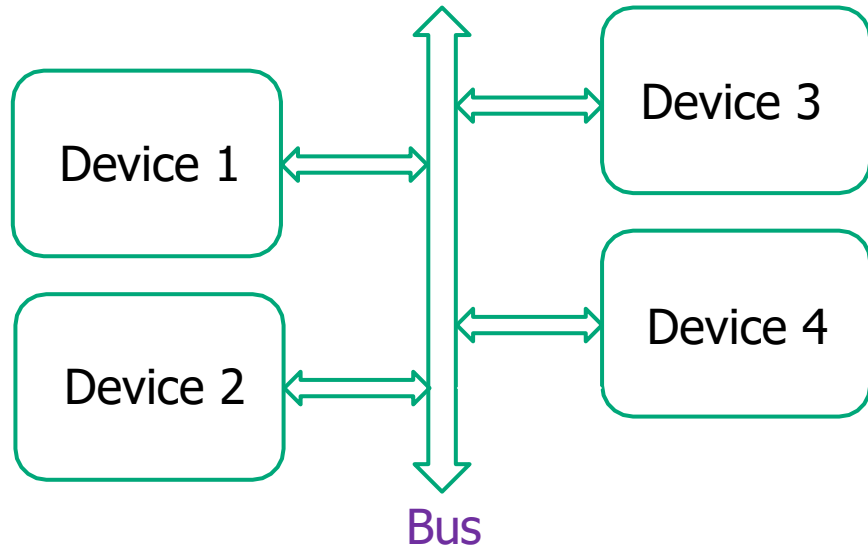
# LIMITATIONS OF ASYNCHRONOUS PROTOCOLS

Receiver in Asynchronous protocol needs to recover timing info from data/control bits (start/stop etc.) – hence, complicated hardware

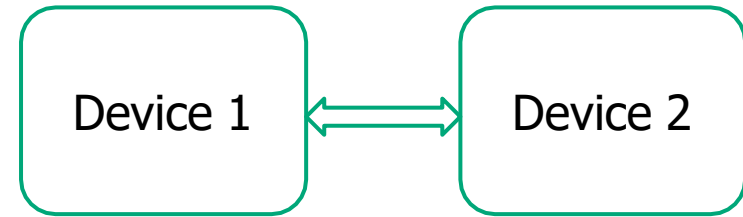


Other means of asynch transmissions, such as read-acknowledge handshaking exist, but require more wires

# BUS-BASED VS POINT-TO-POINT

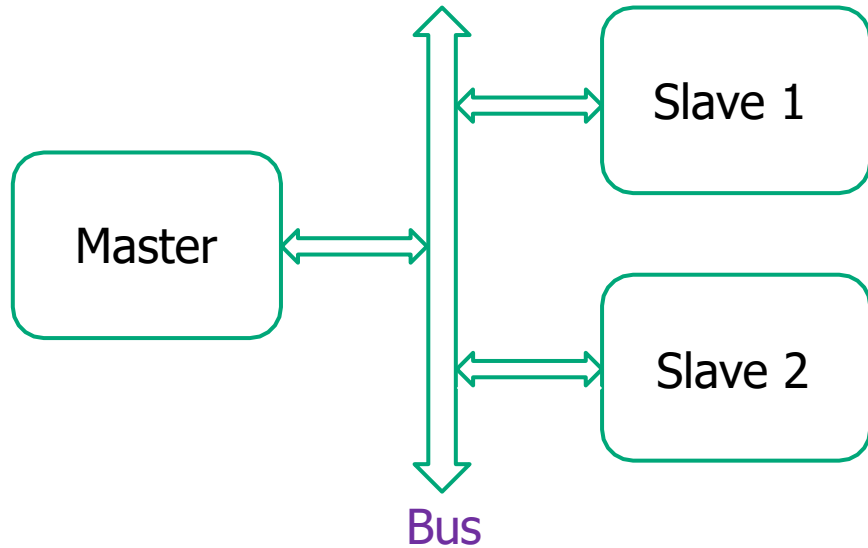


- **Bus:** several devices are connected to the same set of wires (called a bus) – each device has a unique address
- I2C, SPI, USB, AXI, CAN



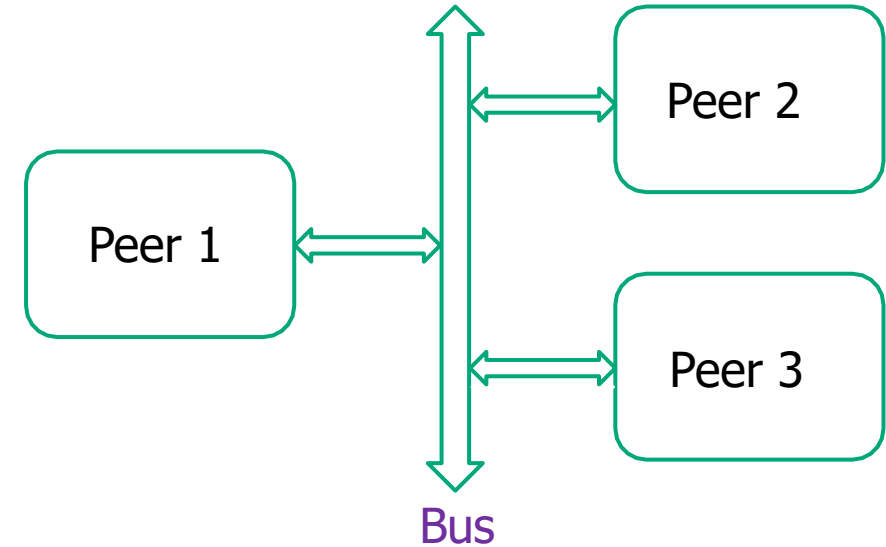
- **Point-to-point:** two devices have a dedicated link between them – no addressing
- UART, AXI-Stream

# MASTER-SLAVE VS PEER-TO-PEER



## Master-slave:

- only master can initiate communication & decides which slave should respond
- master generates the clock (in some synchronous protocols)
- I2C, SPI, USB, AXI, AXI-Stream



## Peer-to-peer:

- any device can initiate communication
- CAN, Ethernet (configuration dependent)

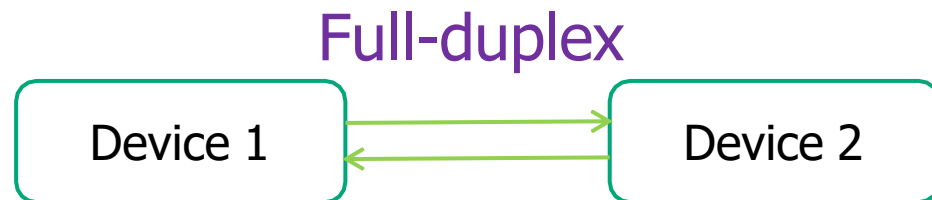
# SIMPLEX VS HALF-DUPLEX VS FULL-DUPLEX



**Simplex:** unidirectional link, one-way interaction  
GPIO output to LEDs, GPIO input from switches, AXI-Stream

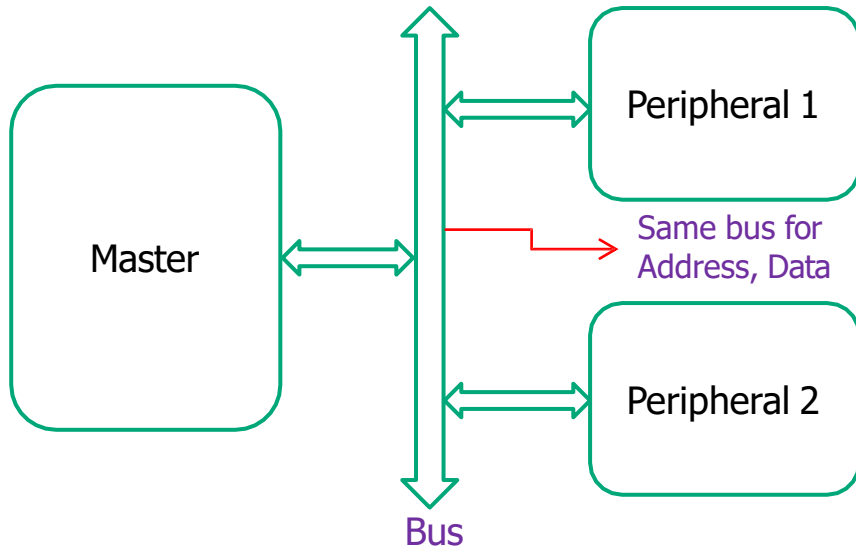


**Half-duplex:** one bi-directional link, devices take turns  
I2C, USB2.0



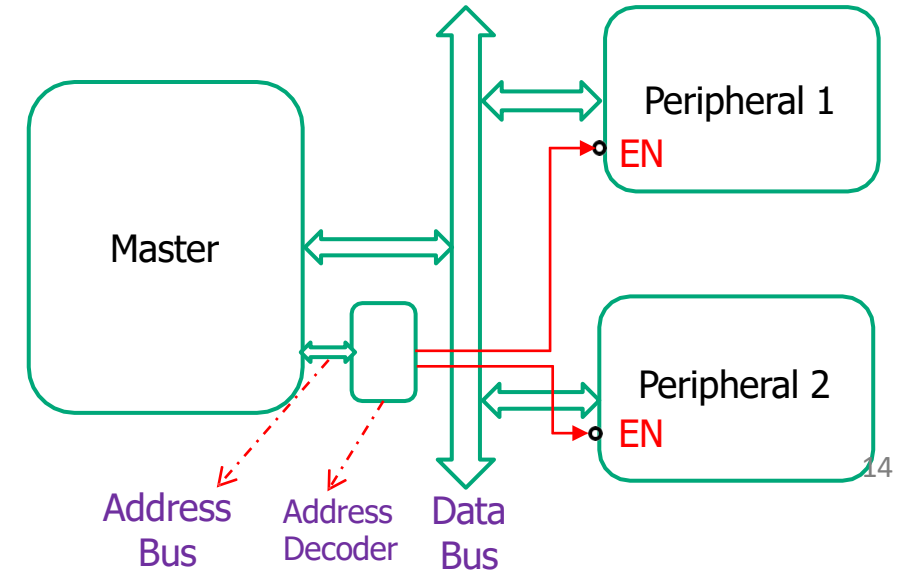
**Full-duplex:** at least two links, can transmit & receive at the same time  
SPI, UART, USB3.x

# IN-BAND VS OUT-OF-BAND ADDRESSING



**In-band:** address of the device being accessed (to select the device), as well as the data (read from or written to the device) are sent through the same bus

- I2C, USB



**Out-of-band:** separate address bus, a decoder enables/activates the device

- SPI, AXI

# CONCLUSION

- What is a Protocol?
- Classification of Protocols
  - Parallel vs Serial
  - Synchronous vs Asynchronous
  - Bus-based vs Point-to-point
  - Master-Slave vs Peer-to-peer
  - Simplex vs Half-duplex vs Full-duplex
  - In-band addressing vs Out-of-band addressing